

TROPIA LIVE

Nền tảng mua sắm thực phẩm tươi sống kết hợp livestream

TÀI LIỆU MÔ TẢ HỆ THỐNG API

Hướng dẫn dành cho người mới: kiến trúc, giao thức livestream và toàn bộ REST API

Stack: Go (Gin) · SRS · PostgreSQL · Redis · Cloudflare R2 · Flutter

Phiên bản API: 1.0.1 — bản cập nhật quy ước response & phân quyền

Mục lục

1. Giới thiệu tổng quan
 - 1.1. Sơ đồ các thành phần
 - 1.2. Quy ước chung của API
 - 1.3. Các API trả về gì? (quy ước response envelope) — **ĐÃ CẬP NHẬT**
 - 1.4. Vai trò người dùng & phân quyền theo role — **MỚI**
2. Các giao thức cần có để livestream
 - 2.1. SRS là gì? · 2.2. Đẩy luồng · 2.3. Phát luồng
 - 2.4. WebSocket realtime · 2.5. Vòng đời một phiên livestream
3. API Xác thực & Tài khoản
 - 3.1. Danh sách endpoint · 3.2. Cơ chế bảo mật đáng chú ý
4. API Phiên Livestream
 - 4.1–4.4. Quản lý phiên, tương tác, host, AI · 4.5. Ví dụ response
5. API Danh mục, Sản phẩm & Cửa hàng
6. API Giỏ hàng, Đơn hàng & Coupon
7. API Thanh toán
8. API Tải ảnh & Webhook SRS
9. Worker nền & hệ thống sự kiện
10. Tóm tắt cho người mới

1. Giới thiệu tổng quan

Tropia là một ứng dụng mua sắm thực phẩm tươi sống tại Việt Nam, có tính năng livestream bán hàng theo phong cách Shopee Live. Người bán (seller) phát trực tiếp, gắn sản phẩm và mã giảm giá vào buổi live, còn người mua (buyer) vừa xem vừa chat, đặt hàng và thanh toán ngay trong phiên live.

Hệ thống gồm một backend viết bằng Go (dùng framework Gin) cung cấp REST API và WebSocket; một media server SRS lo việc nhận và phát luồng video; cùng các thành phần phụ trợ là PostgreSQL (cơ sở dữ liệu), Redis (cache, hàng đợi sự kiện, rate-limit) và Cloudflare R2 (lưu video VOD và hình ảnh).

Tài liệu này được viết cho người mới tiếp cận dự án. Phần đầu giải thích các giao thức cần thiết để livestream (RTMP, SRT, WHIP, HLS, WHEP, WebSocket) một cách dễ hiểu. Phần sau mô tả toàn bộ các nhóm API của hệ thống theo từng module.

1.1. Sơ đồ các thành phần

Thành phần	Vai trò
Flutter App	Ứng dụng đa nền tảng (Android, iOS, Web, Desktop) cho cả người mua và người bán.
Go API (Gin)	Xử lý nghiệp vụ: xác thực, sản phẩm, giỏ hàng, đơn hàng, thanh toán, quản lý phiên live, chat.
Go Worker	Tiến trình nền: gửi email giao dịch và xử lý video VOD (ghi hình → ghép phụ đề chat → tải lên R2).
SRS 5	Media server: nhận luồng video (RTMP/SRT/WHIP) và phát lại cho người xem (HLS/WHEP).
PostgreSQL	Cơ sở dữ liệu chính (người dùng, sản phẩm, đơn hàng, phiên live...).
Redis 7	Cache, khóa phân tán, rate-limit (cửa sổ trượt) và hàng đợi sự kiện (Redis Streams).
Cloudflare R2	Lưu trữ tương thích S3 cho video VOD và hình ảnh tải lên.
Nginx (HLS proxy)	Proxy giữ kết nối keep-alive trước SRS để trình duyệt không cạn cổng TCP khi xem lâu.

1.2. Quy ước chung của API

- Tất cả endpoint nghiệp vụ nằm dưới tiền tố `/api`.
- Xác thực: dùng JWT. Client gửi access token qua header `Authorization: Bearer <token>`.
- Refresh token: lưu trong cookie `HttpOnly refresh_token` (đường dẫn `/api/auth`, `SameSite=Strict`), đồng thời cũng trả về trong body cho app mobile không dùng được cookie.
- Định dạng: request và response đều là JSON (trừ upload ảnh dùng `multipart/form-data`).
- Lỗi: HTTP status phản ánh loại lỗi (401, 403, 404, 409, 422, 429...). Hình dạng body lỗi cuối cùng xem ở mục 1.3.
- Phòng vệ BOLA: khi truy cập tài nguyên không thuộc về mình, hệ thống có thể trả 404 (không phải 403) để không lộ sự tồn tại của tài nguyên.
- Rate limit: giới hạn theo cửa sổ trượt bằng Redis cho các luồng nhạy cảm (đăng nhập, đăng ký, đặt hàng, thanh toán, chat, like).

1.3. Các API trả về gì? (quy ước response envelope) — ĐÃ CẬP NHẬT

Vì sao thay đổi: Mục này đã được viết lại cho khớp với mã nguồn thực tế (internal/http/envelope.go). Bản tài liệu trước mô tả response là JSON "trần" (ví dụ { "session": {...} }), nhưng trên thực tế MỌI response JSON đều bị bọc thêm một lớp "envelope" chuẩn. Hãy đọc kỹ phần này để hiểu hình dạng dữ liệu thật mà client nhận được.

Một middleware tên **EnvelopeWrapper** được đăng ký ở tầng ngoài cùng. Handler vẫn trả payload tự nhiên của nó (ví dụ { "session": {...} } hay { "data": [...] }); middleware đệm (buffer) body đó lại, đặt vào khóa data, suy ra các trường trạng thái từ mã HTTP, rồi phát lại theo khuôn cố định sau:

```
{
  "Result": true,
  "StatusCode": "200",
  "StatusMess": "Success",
  "status": 200,
  "message": "Success",
  "data": { ... } | [ ... ] | null
}
```

Ý nghĩa từng trường của envelope:

- **Result** (bool): true nếu HTTP status thuộc lớp 2xx-3xx (thành công), false nếu ≥ 400 (lỗi).
- **StatusCode** (chuỗi) và **status** (số): cùng là mã HTTP, một bản dạng chuỗi, một bản dạng số nguyên.
- **StatusMess** và **message**: cùng một thông điệp người đọc được. Thứ tự ưu tiên khi suy ra: (1) message do handler chủ động đặt qua SetMessage — ví dụ "Đăng nhập thành công"; (2) khóa message trong payload khi thành công; (3) khóa error trong body khi lỗi; (4) thông điệp mặc định theo mã HTTP (ví dụ 403 → "Không có quyền truy cập", 404 → "Không tìm thấy").
- **data**: payload gốc do handler trả về. Khi thành công, toàn bộ payload nằm trong data. Khi lỗi, data = null.

Thành công vs. lỗi

Khi **thành công** (2xx/3xx): payload của handler được giữ nguyên và đặt vào data. Nếu payload có khóa message, giá trị đó được nâng lên StatusMess/message (vẫn nằm trong data).

Khi **lỗi** (≥ 400): bộ xử lý lỗi tập trung ErrorHandler sinh ra body dạng { "error": "...", "code": "...", "details": ... }. Envelope nâng chuỗi error lên thành message, đồng thời **đặt** data = null.

Quan trọng: Ở phiên bản envelope hiện tại, chỉ thông điệp lỗi (error) được giữ lại trong message. Các trường code và details mà handler sinh ra **không xuất hiện** ở response cuối cùng (envelope không có khóa code/details ở mức trên cùng). Vì vậy, cột mã lỗi (ví dụ STREAM_NOT_FOUND, SELLER_ONLY) trong các bảng "Response của TẤT CẢ endpoint..." bên dưới là **mã nội bộ để tra cứu/đối soát log**, còn client thực tế chỉ nhận được message + status. Nếu cần trả code cho client, phải bổ sung trường vào struct Envelope và buildEnvelope.

Hành động không có dữ liệu (trước đây ghi là 204 No Content)

Handler các thao tác như join/leave/like/logout gọi c.Status(204) và không ghi body.

EnvelopeWrapper vẫn bọc lại: HTTP status giữ nguyên **204** nhưng kèm theo một envelope với data: null, Result: true (được xác nhận bằng unit test TestEnvelope_NoContent). Một số client

bỏ qua body khi gặp 204 — hãy dựa vào status/Result, không dựa vào việc body rỗng.

Những response KHÔNG bị bọc envelope

skipEnvelope cho đi thẳng (không bọc) các trường hợp: nâng cấp WebSocket (Upgrade: websocket); /health; /metrics; mọi đường dẫn /docs/...; webhook SRS /api/srs/... (trả số trần 0/1); đường dẫn chứa /hls/ hoặc /ws/; file kết thúc .m3u8 hoặc .ts; các redirect 3xx (giữ header Location); và mọi body không phải application/json (file, text) — đi qua nguyên trạng.

Hai ví dụ envelope đầy đủ

Đăng nhập thành công — handler trả {access_token, refresh_token, user...}, envelope bọc lại thành:

```
POST /api/auth/login → HTTP 200
{
  "Result": true, "StatusCode": "200", "StatusMess": "Đăng nhập thành công",
  "status": 200, "message": "Đăng nhập thành công",
  "data": {
    "access_token": "eyJhbGciOi...", "refresh_token": "f3a9c2e1-...",
    "token_type": "Bearer", "expires_in": 900,
    "user": { "id": "8c1d...", "role": "buyer", "email_verified": true }
  }
}
```

Không tìm thấy phiên live — handler c.Error(NewNotFound(...)), envelope bọc lại thành:

```
GET /api/live/streams/:id → HTTP 404
{
  "Result": false, "StatusCode": "404",
  "StatusMess": "không tìm thấy phiên live",
  "status": 404, "message": "không tìm thấy phiên live",
  "data": null
}
```

Cách đọc các ví dụ bên dưới: Để dễ đọc, từ mục 3 trở đi các khối "Ví dụ response" và bảng "Response của TẤT CẢ endpoint..." chỉ hiển thị **phần nằm trong khóa data** (payload gốc của handler). Hãy luôn ngầm hiểu rằng toàn bộ được bọc trong envelope chuẩn ở trên, và thông điệp lỗi được nâng lên message.

Các mã trạng thái HTTP hay gặp:

Mã	Ý nghĩa	Khi nào gặp
400	Bad Request	Dữ liệu gửi lên sai định dạng / thiếu trường / không qua validation.
401	Unauthorized	Thiếu token, token hết hạn, hoặc sai email/mật khẩu khi đăng nhập.
403	Forbidden	Đã đăng nhập nhưng không đủ quyền (ví dụ buyer gọi API của seller), hoặc không phải chủ tài nguyên.
404	Not Found	Không tìm thấy tài nguyên — hoặc cố tình giấu để phòng vệ BOLA (xem đơn của người khác).
409	Conflict	Xung đột trạng thái: hết kho, email đã tồn tại, coupon đã dùng.
422	Unprocessable	Qua được validation cơ bản nhưng vi phạm quy tắc nghiệp vụ.
429	Too Many Requests	Vượt rate limit (đăng nhập, đặt hàng, chat, like...).
500	Server Error	Lỗi không lường trước phía server.

Ví dụ một lỗi validation 400 ở mức handler (trước khi bọc envelope). Lưu ý details chỉ tồn tại ở tầng handler; sau khi qua envelope, client chỉ thấy message:

```
{ "error": "dữ liệu không hợp lệ",
  "code": "VALIDATION_ERROR",
  "details": {
    "email": "email không đúng định dạng",
    "password": "mật khẩu tối thiểu 8 ký tự" } }
```

1.4. Vai trò người dùng & phân quyền theo role — MỚI

MỚI: Mục này là phần bổ sung mới, giới thiệu mô hình phân quyền theo vai trò (role-based access control) và luồng kiểm tra quyền chạy trên mỗi request. Nội dung lấy trực tiếp từ `internal/auth` (`jwt.go`, `middleware.go`, `service.go`) và phần đăng ký route trong `cmd/api/main.go`.

Hệ thống có đúng **ba vai trò**, định nghĩa trong `auth.Role` (`jwt.go`):

Role	Mô tả	Nhóm quyền tiêu biểu
buyer	Người mua — vai trò mặc định khi đăng ký.	Xem live/sản phẩm/cửa hàng; join/leave/like/chat trong phiên; giỏ hàng; đặt hàng & thanh toán; theo dõi cửa hàng; gọi AI gợi ý câu hỏi và được bot trả lời.
seller	Người bán (kế thừa toàn bộ quyền của buyer).	Tạo & kết thúc phiên live; tạo/sửa sản phẩm (quick-create, đổi trạng thái); tạo & sửa cửa hàng của mình; tải ảnh sản phẩm/biến thể/shop/live; phân tích AI phiên (analyze).
admin	Quản trị — vượt qua mọi <code>RequireRole</code> .	Quản lý danh mục (tạo/sửa/xóa), thêm giá trị thuộc tính; và thay mặt seller thao tác trên mọi tài nguyên (admin luôn nằm trong tập cho phép của guard seller và guard chủ-sở-hữu).

Luồng kiểm tra quyền trên mỗi request

```
Request
| Authorization: Bearer <access_token>
▼
[1] auth.Middleware – xác minh chữ ký HS256, gởi mã Claims{UserID, Role},
đặt claims vào gin.Context. (thiếu/sai/hết hạn → 401)
▼
[2] auth.RequireRole(...) – guard theo VAI TRÒ (nếu route có gñn).
role không nằm trong tập cho phép → 403
▼
[3] Service layer – kiểm tra QUYỀN SỞ HỮU tài nguyên (chỉ phiên/shop/
sản phẩm/đơn). Không phải chủ → 403 NOT_*_OWNER,
hoặc 404 (BOLA) với đơn hàng đã giao sự tñn tại.
▼
Handler → trỏ payload → (được bọc envelope ở mục 1.3)
```

Bốn tầng guard được khai báo sẵn trong `main.go` và gắn vào từng nhóm route:

Guard (main.go)	Định nghĩa	Dùng cho
authMw	<code>auth.Middleware(jwtSvc)</code>	Mọi route chỉ cần ĐÃ ĐĂNG NHẬP (bất kỳ role nào): giỏ hàng, đơn hàng, chat, follow, join/leave...
sellerMw	<code>RequireRole(seller, admin)</code>	API người bán: tạo phiên live, quick-create sản phẩm, tạo shop, upload ảnh, analyze...

Guard (main.go)	Định nghĩa	Dùng cho
adminMw	RequireRole(admin)	API quản trị: tạo/sửa/xóa danh mục, thêm giá trị thuộc tính.
optAuthMw	optionalAuth.jwtSvc)	Xác thực TÙY CHỌN: có token thì đọc danh tính, không có vẫn cho qua (vd GET /api/shops/:slug để biết is_following).

Phân biệt quyền theo VAI TRÒ và quyền theo SỞ HỮU

Đây là điểm hay nhầm. RequireRole chỉ kiểm tra bạn có **đúng loại tài khoản** hay không. Kể cả một seller hợp lệ, nếu thao tác trên tài nguyên **không phải của mình** (phiên live, cửa hàng, sản phẩm hay đơn của người khác) thì vẫn bị chặn ở tầng service bằng kiểm tra chủ sở hữu. Các mã lỗi liên quan:

- **401 UNAUTHORIZED** — thiếu/sai/hết hạn token (chặn ở auth.Middleware).
- **403 — SELLER_ONLY / ADMIN_ONLY / FORBIDDEN** — token hợp lệ nhưng role không đủ (chặn ở RequireRole).
- **403 — NOT_STREAM_OWNER / NOT_SHOP_OWNER / NOT_PRODUCT_OWNER / NOT_ORDER_OWNER** — đúng role nhưng không phải chủ tài nguyên (chặn ở service).
- **404 (BOLA)** — cố tình giấu tài nguyên của người khác, điển hình là đơn hàng (ORDER_NOT_FOUND) để không lộ rằng đơn đó tồn tại.

Lưu ý: Chống leo thang quyền khi đăng ký (mass assignment): API register chỉ chấp nhận role buyer hoặc seller. Mọi giá trị khác — kể cả admin — bị âm thầm hạ về buyer (xem service.go). Tài khoản admin chỉ được tạo trực tiếp trong cơ sở dữ liệu, không bao giờ qua API công khai.

2. Các giao thức cần có để livestream

Một buổi livestream gồm hai chiều: đẩy luồng (ingest) từ người bán lên server, và phát luồng (playback) từ server xuống người xem. Mỗi chiều có thể dùng nhiều giao thức khác nhau tùy thiết bị. Dưới đây là các giao thức Tropia hỗ trợ.

2.1. SRS là gì?

SRS (Simple Realtime Server) là một media server mã nguồn mở — tức là một máy chủ chuyên xử lý video/âm thanh thời gian thực. Trong Tropia, SRS đóng vai trò trung gian giữa người bán và người xem: nó nhận luồng video do host đẩy lên (qua RTMP/SRT/WHIP), rồi phát lại luồng đó cho hàng loạt người xem dưới nhiều định dạng khác nhau (HLS/HTTP-FLV/WHEP).

Hãy hình dung SRS như một đài truyền hình thu nhỏ: host là người quay phim gửi tín hiệu vào đài, đài chuyển đổi tín hiệu sang các 'kênh' phù hợp với từng loại tivi, và khán giả chỉ việc mở kênh để xem. Nhờ vậy, backend Go không cần tự xử lý từng khung hình video — đây là việc rất nặng — mà chỉ quản lý thông tin phiên live và để SRS lo phần truyền dẫn.

Vì sao Tropia chọn SRS:

- Đa giao thức: nhận RTMP, SRT, WHIP và phát HLS, HTTP-FLV, WHEP — host và người xem dùng thiết bị nào cũng được phục vụ.
- Tự chuyển đổi: SRS tự động chuyển một luồng RTMP đầu vào thành nhiều định dạng đầu ra, kể cả WebRTC, mà không cần cấu hình phức tạp.
- Webhook (HTTP callback): SRS chủ động gọi về backend khi có sự kiện (bắt đầu phát, ngắt phát, ghi hình xong...). Nhờ đó backend biết khi nào đánh dấu phiên 'live', khi nào xử lý video xem lại. Xem chi tiết ở mục 8.2.
- DVR (ghi hình): SRS tự ghi lại buổi live ra file FLV để sau đó worker dựng thành video xem lại (VOD).

Các cổng mặc định của SRS: 1935 (RTMP), 8080 (HLS/FLV thô — người xem đi qua Nginx ở 8090), 1985 (HTTP API + WHIP/WHEP), 8000/udp (WebRTC), 10080/udp (SRT).

Lưu ý: SRS chạy trong một container riêng (xem `infra/docker-compose.yml` và `k8s/srs.yaml`), tách biệt hoàn toàn với backend Go. Hai bên chỉ giao tiếp qua URL stream và webhook — backend không bao giờ đụng vào dữ liệu video.

2.2. Giao thức ĐẨY luồng (người bán → SRS)

Giao thức	Giải thích cho người mới
RTMP	Real-Time Messaging Protocol. Giao thức kinh điển, được hầu hết phần mềm phát sóng hỗ trợ (OBS Studio, Larix Broadcaster). Đơn giản, ổn định, độ trễ vừa phải. Đây là cách phổ biến nhất để host đẩy hình.
SRT	Secure Reliable Transport. Giao thức hiện đại trên nền UDP, chịu mất gói tốt, phù hợp đường truyền không ổn định (3G/4G ngoài trời). Dùng cho host chuyên nghiệp cần chất lượng cao.
WHIP	WebRTC-HTTP Ingestion Protocol. Cho phép đẩy luồng trực tiếp từ trình duyệt hoặc trong app (camera điện thoại) qua WebRTC, độ trễ cực thấp, không cần phần mềm ngoài.

Khi người bán tạo phiên live (gọi POST `/api/live/streams`), backend sinh một stream key và trả về cả ba URL đẩy luồng (RTMP, WHIP, SRT) để host chọn dùng cái nào tùy thiết bị.

```
rtmp://<host>:1935/live/<stream_key>
http://<host>:1985/rtc/v1/whip/?app=live&stream=<stream_key>
srt://<host>:10080?streamid=#!::r=live/<stream_key>,m=publish
```


2.3. Giao thức PHÁT luồng (SRS → người xem)

Giao thức	Giải thích cho người mới
HLS	HTTP Live Streaming. Cắt video thành các đoạn nhỏ (.ts) và một playlist (.m3u8). Tương thích rộng rãi với mọi trình duyệt và thiết bị di động. Độ trễ ~5-8 giây nhưng rất ổn định — đây là kênh phát chính cho người mua trong app Flutter.
HTTP-FLV	Phát video FLV qua HTTP. Độ trễ thấp hơn HLS đôi chút, dùng làm phương án thay thế.
WHEP	WebRTC-HTTP Egress Protocol. Phát qua WebRTC, độ trễ siêu thấp (gần thời gian thực), phù hợp khi cần tương tác tức thì.

Khi người xem gọi GET /api/live/streams/:id/playback, backend trả về URL HLS/FLV/WHEP. App Flutter dùng widget HlsViewer (video_player + chewie) để phát HLS.

Lưu ý: URL HLS không trở thẳng vào cổng 8080 của SRS mà đi qua Nginx proxy ở cổng 8090. Lý do: SRS gửi 'Connection: close' trên mọi response nên trình duyệt mở một kết nối TCP mới cho từng đoạn .ts, gây cạn pool cổng tạm sau ~10 phút xem. Nginx giữ keep-alive để khắc phục.

2.4. WebSocket cho realtime

Ngoài video, buổi live còn cần cập nhật realtime cho chat, số liệu (viewer/like/follow), ghim sản phẩm và phát mã giảm giá. Tropia dùng WebSocket cho việc này:

- GET /api/live/streams/:id/ws/chat — kênh WebSocket theo từng phiên: server đẩy tin nhắn chat mới, sự kiện cập nhật số liệu, ghim sản phẩm, phát coupon. Client chỉ nhận (đọc), khi gửi tin vẫn POST qua REST.
- GET /api/live/ws/list — kênh WebSocket toàn nền tảng: báo cho tab Live biết danh sách phiên đã thay đổi (có phiên mới mở/kết thúc) để client tự gọi lại danh sách.

2.5. Vòng đời một phiên livestream

- **1. Tạo phiên:** host gọi POST /api/live/streams → backend tạo bản ghi phiên (trạng thái 'scheduled'), sinh stream key và trả về các URL đẩy luồng.
- **2. Đẩy luồng:** host dùng OBS/Larix đẩy RTMP (hoặc WHIP từ app) vào SRS.
- **3. Webhook on_publish:** SRS gọi POST /api/srs/on_publish → backend đánh dấu phiên 'live'.
- **4. Người xem vào:** buyer gọi GET /api/live/streams/:id/playback để lấy URL HLS và bắt đầu xem; gọi POST /:id/join để được tính là viewer.
- **5. Tương tác:** chat, like, theo dõi, thêm giỏ hàng, đặt hàng ngay trong live; host ghim sản phẩm và phát coupon — tất cả được đẩy realtime qua WebSocket.
- **6. Kết thúc:** host gọi POST /:id/end → backend đặt trạng thái 'ended'. SRS gọi on_unpublish khi ngắt luồng.
- **7. Xử lý VOD:** SRS ghi hình ra file FLV và gọi on_dvr → backend phát sự kiện 'recording.created' → worker ghép phụ đề chat + overlay, chuyển sang MP4, tải lên R2 và lưu URL để xem lại.

3. API Xác thực & Tài khoản

Nhóm `/api/auth` lo việc đăng ký, xác thực OTP qua email, đăng nhập, làm mới token, đăng xuất, quên/đặt lại mật khẩu và đăng nhập bằng Google OAuth2.

3.1. Danh sách endpoint

Meth od	Đường dẫn	Quyền	Mô tả
POST	<code>/api/auth/register</code>	Công khai	Đăng ký tài khoản buyer/seller, mã hóa mật khẩu bằng bcrypt (cost 12), sinh OTP 6 số lưu vào Redis và gửi email. Giới hạn 3 lần / 5 phút.
POST	<code>/api/auth/verify-otp</code>	Công khai	Xác thực email bằng OTP 6 số. Khi thành công, đánh dấu email_verified và phát sự kiện gửi email chào mừng.
POST	<code>/api/auth/resent-verify-email</code>	Công khai	Gửi lại OTP. Chống dò tài khoản: luôn trả cùng một thông báo.
POST	<code>/api/auth/login</code>	Công khai	Đăng nhập bằng email + mật khẩu. Giới hạn 5 lần / phút; khóa 15 phút sau 5 lần sai. Trả về access token + refresh token.
POST	<code>/api/auth/refresh</code>	Cookie/Body	Xoay vòng refresh token và cấp access token mới. Nếu phát hiện token bị tái sử dụng, thu hồi toàn bộ 'family' token (chống đánh cắp).
POST	<code>/api/auth/logout</code>	Cookie	Thu hồi một refresh token và xóa cookie.
POST	<code>/api/auth/logout-all</code>	JWT	Thu hồi tất cả refresh token của người dùng (đăng xuất mọi thiết bị).
GET	<code>/api/auth/me</code>	JWT	Trả về thông tin hồ sơ người dùng hiện tại.
POST	<code>/api/auth/forgot-password</code>	Công khai	Sinh token đặt lại mật khẩu (hiệu lực 1 giờ) và gửi email. Luôn trả 200 để chống dò tài khoản.
POST	<code>/api/auth/reset-password</code>	Công khai	Đặt lại mật khẩu bằng token, đồng thời thu hồi mọi refresh token cũ.
GET	<code>/api/auth/google</code>	Công khai	Chuyển hướng tới Google OAuth2 (hỗ trợ PKCE).
GET	<code>/api/auth/google/callback</code>	Công khai	Google gọi lại sau khi đăng nhập; tạo One-Time-Code (TTL 60s) rồi deep-link về app.
GET	<code>/api/auth/google/exchange</code>	Công khai	Đổi One-Time-Code (dùng một lần) lấy access token; xác minh PKCE nếu có.

Bổ sung: Ngoài ra có alias theo spec: POST `/api/login` nhận {username (SĐT/email), password} và trả {token, expires, user{...}} — tiện cho client cũ. Mọi response của nhóm auth vẫn được bọc envelope như mục 1.3.

Ví dụ response (phần data bên trong envelope)

```
POST /api/auth/login → 200 OK
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "refresh_token": "f3a9c2e1-7b04-4d...",
  "token_type": "Bearer", "expires_in": 900,
  "user": { "id": "8c1d...e2", "email": "ban@vidu.vn",
  "name": "Nguyễn Văn A", "role": "buyer",
  "email_verified": true } }
```

```
GET /api/auth/me → 200 OK
{ "user": { "id": "8c1d...e2", "email": "ban@vidu.vn",
"name": "Nguyễn Văn A", "role": "buyer",
"avatar_url": "https://cdn.../a.webp",
"created_at": "2025-11-02T09:14:22Z" } }
```

```
POST /api/auth/register → 201 Created
{ "user": { "id": "8c1d...", "email": "ban@vidu.vn", "role": "buyer",
"email_verified": false } } // dev: kèm "otp_dev"
```

```
POST /api/auth/login → 401 (data: null, message lấy từ error)
handler: { "error": "email hoặc mật khẩu không đúng",
"code": "INVALID_CREDENTIALS" }
```

Lưu ý: Đăng nhập sai trả 401 với message "email hoặc mật khẩu không đúng"; quá 5 lần sai khóa 15 phút (429). role trong hồ sơ là một trong buyer/seller/admin (xem mục 1.4).

Response của TẤT CẢ endpoint xác thực

Cách đọc mã HTTP: Quy ước mã HTTP trong các bảng từ đây: cột "Thành công" ghi mã 2xx/3xx + phần data; cột "Các mã lỗi có thể gặp" liệt kê MỌI tình huống lỗi của endpoint đó kèm mã HTTP. Để khởi lập lại: **mọi endpoint** đều có thể trả **500 INTERNAL_ERROR** (lỗi server) và **400 VALIDATION_ERROR** (body/tham số sai); mã **401 UNAUTHORIZED** chỉ được ghi ở endpoint bắt buộc đăng nhập. Tên CODE là mã nội bộ (không nằm trong envelope cuối — xem 1.3).

Endpoint	Thành công	Các mã lỗi có thể gặp
POST /register	201 { user }	400 VALIDATION_ERROR · 409 EMAIL_EXISTS · 429 RATE_LIMITED
POST /verify-otp	200 { access_token, refresh_token, user }	400 INVALID_OTP · 422 VALIDATION_ERROR · 429 RATE_LIMITED
POST /resend-verify-email	200 { message, expires_in }	429 RATE_LIMITED
POST /login	200 { access_token, refresh_token, user }	401 INVALID_CREDENTIALS · 429 ACCOUNT_LOCKED · 429 RATE_LIMITED
POST /refresh	200 { access_token, refresh_token }	401 INVALID_REFRESH_TOKEN
POST /logout	204 (data:null)	401 UNAUTHORIZED
POST /logout-all	204 (data:null)	401 UNAUTHORIZED
GET /me	200 { user }	401 UNAUTHORIZED
POST /forgot-password	200 { message }	429 RATE_LIMITED
POST /reset-password	200 { message }	400 INVALID_RESET_TOKEN · 422 VALIDATION_ERROR
GET /google	302 → Google	302/500 (cấu hình OAuth sai)
GET /google/callback	302 → deep-link + OTC	302 OAUTH_FAILED
GET /google/exchange	200 { access_token, refresh_token, user }	400 INVALID_OTC

Lưu ý: forgot-password luôn trả 200 + message thành công dù email có tồn tại hay không, để không lộ email nào đã đăng ký (chống dò tài khoản).

3.2. Cơ chế bảo mật đáng chú ý

- **Xoay vòng & phát hiện tái sử dụng refresh token:** mỗi lần refresh sẽ thu hồi token cũ và cấp token mới trong cùng 'family'. Nếu một token đã bị thu hồi được dùng lại, cả family bị vô hiệu hóa — phòng trường hợp token bị đánh cắp.
- **Chống leo thang quyền (Mass Assignment):** khi đăng ký, chỉ chấp nhận role 'buyer' hoặc 'seller'. Giá trị 'admin' bị âm thầm hạ về 'buyer'. Admin chỉ được tạo trực tiếp ở DB. (Xem thêm mục 1.4.)
- **PKCE cho OAuth mobile:** dùng code_challenge/code_verifier (S256) để chống ứng dụng độc chiếm deep-link callback và đánh cắp mã đăng nhập.
- **Khóa tài khoản:** sau 5 lần đăng nhập sai, tài khoản bị khóa 15 phút.

4. API Phiên Livestream

Nhóm `/api/live` là trái tim của hệ thống: tạo/kết thúc phiên, lấy URL phát, danh sách phiên, chat, số liệu, ghim sản phẩm, coupon trong live, bật/tắt bot AI và timeline để xem lại VOD.

4.1. Quản lý phiên & phát luồng

Meth od	Đường dẫn	Quyền	Mô tả
GET	<code>/api/live/streams</code>	Công khai	Danh sách phiên đang live (cache Redis 3s, lấy kèm sản phẩm ghim theo lô để tránh N+1). Stream key bị ẩn.
GET	<code>/api/live/streams/:id</code>	Công khai	Chi tiết một phiên (đã ẩn stream key).
POST	<code>/api/live/streams</code>	Seller/Admin	Tạo phiên mới, trả về các URL đẩy luồng (RTMP/WHIP/SRT).
POST	<code>/api/live/streams/:id/end</code>	Chủ phiên/Admin	Kết thúc phiên (status='ended' vĩnh viễn).
GET	<code>/api/live/streams/:id/playback</code>	Công khai	Trả về URL phát HLS/FLV/WHEP.
GET	<code>/api/live/streams/:id/publish</code>	Chủ phiên/Admin	Trả về URL đẩy luồng kèm stream key (chỉ chủ phiên xem được).
GET	<code>/api/live/streams/:id/stats</code>	Công khai	Số liệu realtime: viewer, like, order, doanh thu, follow, sản phẩm đang ghim.

4.2. Tương tác của người xem

Meth od	Đường dẫn	Quyền	Mô tả
POST	<code>/api/live/streams/:id/join</code>	JWT	Ghi nhận người xem tham gia (idempotent). Host không tự tính là viewer của chính mình.
POST	<code>/api/live/streams/:id/leave</code>	JWT	Người xem rời phiên.
POST	<code>/api/live/streams/:id/like</code>	Công khai	Tăng số like. Giới hạn 60 lần / phút / IP để chống bot thổi phồng.
GET	<code>/api/live/streams/:id/chat</code>	Công khai	Lấy tối đa 50-100 tin chat gần nhất.
POST	<code>/api/live/streams/:id/chat</code>	JWT	Gửi tin nhắn chat (giới hạn 30 / phút / người). Có thể kích hoạt bot AI tự trả lời.
POST	<code>/api/live/streams/:id/track-cart-add</code>	JWT	Đếm lượt thêm vào giỏ trong live.
POST	<code>/api/live/streams/:id/track-follow</code>	JWT	Ghi nhận lượt theo dõi (dedupe theo người dùng).

4.3. Quản lý của host trong phiên

Metho d	Đường dẫn	Quyền	Mô tả
PATCH	<code>/api/live/streams/:id/bot</code>	Chủ phiên/Admin	Bật/tắt bot AI tự trả lời câu hỏi viewer.
GET	<code>/api/live/streams/:id/products</code>	Công khai	Danh sách sản phẩm gắn vào phiên.
POST	<code>/api/live/streams/:id/products</code>	Chủ phiên/Admin	Thêm sản phẩm vào phiên.

Method	Đường dẫn	Quyền	Mô tả
PUT	/api/live/streams/:id/products	Chủ phiên/Admin	Thay toàn bộ danh sách sản phẩm (khi host quay lại từ màn chọn sản phẩm).
DELETE	/api/live/streams/:id/products/:productid	Chủ phiên/Admin	Gỡ một sản phẩm khỏi phiên.
POST	/api/live/streams/:id/pin	Chủ phiên/Admin	Ghim một sản phẩm đang giới thiệu (kiểu 'GẤP LÊN' của Shopee Live); body rỗng để bỏ ghim.
POST	/api/live/streams/:id/coupons	Chủ phiên	Tạo coupon gắn với phiên; tự động phát banner tới mọi viewer.
POST	/api/live/streams/:id/coupons/:couponid/announce	Chủ phiên	Phát lại một coupon đã có giữa phiên.
GET	/api/live/streams/:id/coupons	Công khai	Danh sách coupon của phiên (cho viewer).
GET	/api/live/streams/:id/timeline	Công khai	Timeline xem lại VOD: gộp sự kiện host + tin chat theo mốc thời gian (stream_offset_ms) để đồng bộ overlay với video MP4.

4.4. AI hỗ trợ live (DeepSeek)

Method	Đường dẫn	Quyền	Mô tả
POST	/api/live/streams/:id/ai-suggestions	JWT	Sinh 3-5 câu hỏi ngắn kiểu người xem (về giá, ship, bảo quản, biến thể, khuyến mãi) dựa trên sản phẩm ghim.
POST	/api/live/streams/:id/ai-reply	JWT	Bot tự trả lời một câu hỏi của viewer (ngắn gọn, tiếng Việt).
POST	/api/live/streams/:id/analyze	Seller/Admin	Phân tích cảm xúc & gợi ý cải thiện cho phiên (sentiment + tóm tắt + 4 mẹo).

Lưu ý: Bot AI có hạn mức 30 lượt trả lời / giờ / phiên (kiểm soát qua Redis) để tránh lạm dụng. Khi viewer hỏi về giảm giá, bot chỉ nhắc đúng mã coupon đang phát, không bịa khuyến mãi không có thật.

4.5. Ví dụ response (phần data bên trong envelope)

```
GET /api/live/streams → 200 OK
{ "data": [ { "id": "alb2...", "title": "Rau sạch Đà Lạt",
"status": "live", "viewer_count": 248,
"shop": { "id": "s9...", "name": "Nông trại Xanh",
"slug": "nong-trai-xanh" },
"started_at": "2026-06-01T03:10:00Z" } ],
"pagination": { "page": 1, "limit": 20, "total": 7 } }
```

```
POST /api/live/streams/:id/publish → 200 OK
{ "stream_key": "alb2c3...(chỉ trỏ cho chủ phiên)",
"ingest": { "rtmp": "rtmp://live.tropia.vn/live",
"srt": "srt://live.tropia.vn:10080?streamid=...",
"whip": ".../rtc/v1/whip/?app=live&stream=alb2" } }
```

```
POST /api/live/streams/:id/like → 204 (envelope data:null)
POST /api/live/streams/:id/publish → 403
message: "bạn không phải chủ phiên live này" (NOT_STREAM_OWNER)
```

Lưu ý: stream_key chỉ xuất hiện trong response của publish dành cho chủ phiên; mọi viewer khác không bao giờ nhận được khóa này.

Response của TẤT CẢ endpoint phiên live

Cột "Thành công" = mã 2xx + data; cột "Các mã lỗi có thể gặp" liệt kê mọi tình huống. Nhắc lại: 500 và 400 VALIDATION_ERROR áp dụng ngầm cho mọi endpoint; chỉ ghi 401 ở endpoint cần đăng nhập.

Endpoint	Thành công	Các mã lỗi có thể gặp
GET /streams	200 { data[], pagination }	– (chỉ 500 khi DB/Redis lỗi)
POST /streams	201 { session }	401 · 403 SELLER_ONLY
GET /streams/:id	200 { session }	404 STREAM_NOT_FOUND
GET /streams/:id/publish	200 { stream_key, ingest }	401 · 403 NOT_STREAM_OWNER · 404 STREAM_NOT_FOUND
GET /streams/:id/playback	200 { hls, flv, whep }	404 STREAM_NOT_FOUND
GET /streams/:id/stats	200 { viewer_count, like_count, ... }	404 STREAM_NOT_FOUND
POST /streams/:id/end	200 { session: status=ended }	401 · 403 NOT_STREAM_OWNER · 404 STREAM_NOT_FOUND
POST /streams/:id/join	204 (data:null)	401 · 404 STREAM_NOT_FOUND
POST /streams/:id/leave	204 (data:null)	401 · 404 STREAM_NOT_FOUND
POST /streams/:id/like	204 (data:null)	404 STREAM_NOT_FOUND · 429 RATE_LIMITED
POST /streams/:id/chat	201 { message }	401 · 403 CHAT_MUTED · 404 STREAM_NOT_FOUND · 429 RATE_LIMITED
GET /streams/:id/chat	200 { messages[] }	404 STREAM_NOT_FOUND
POST /streams/:id/track-cart-add	204 (data:null)	401 · 404 STREAM_NOT_FOUND
POST /streams/:id/track-follow	204 (data:null)	401 · 404 STREAM_NOT_FOUND
PATCH /streams/:id/bot	200 { ai_bot_enabled }	401 · 403 NOT_STREAM_OWNER · 404 STREAM_NOT_FOUND
GET /streams/:id/products	200 { products[] }	404 STREAM_NOT_FOUND
POST /streams/:id/products	200 { products[] }	401 · 403 NOT_STREAM_OWNER · 404 · 409 OUT_OF_STOCK
PUT /streams/:id/products	200 { products[] }	401 · 403 NOT_STREAM_OWNER · 404 STREAM_NOT_FOUND
DELETE /streams/:id/products/:pid	204 (data:null)	401 · 403 NOT_STREAM_OWNER · 404
POST /streams/:id/pin	200 { pinned_product_id } (ràng buộc ghim)	401 · 403 NOT_STREAM_OWNER · 404

Endpoint	Thành công	Các mã lỗi có thể gặp
POST /streams/:id/coupons	201 { coupon }	401 · 403 NOT_STREAM_OWNER · 404 · 422 VALIDATION_ERROR
POST /streams/:id/coupons/:cid/announce	204 (data:null)	401 · 403 NOT_STREAM_OWNER · 404 COUPON_NOT_FOUND
GET /streams/:id/coupons	200 { coupons[] }	404 STREAM_NOT_FOUND
GET /streams/:id/timeline	200 { events[] }	404 STREAM_NOT_FOUND
POST /streams/:id/ai-suggestions	200 { suggestions[] }	401 · 404 · 429 BOT_QUOTA_EXCEEDED
POST /streams/:id/ai-reply	200 { reply }	401 · 404 · 429 BOT_QUOTA_EXCEEDED
POST /streams/:id/analyze	200 { sentiment, summary, tips[] }	401 · 403 SELLER_ONLY · 404

5. API Danh mục, Sản phẩm & Cửa hàng

5.1. Danh mục — /api/categories

Method	Đường dẫn	Quyền	Mô tả
GET	/api/categories	Công khai	Danh sách danh mục (cache Redis 5 phút).
GET	/api/categories/:slug	Công khai	Chi tiết một danh mục theo slug.
POST	/api/categories	Admin	Tạo danh mục mới.
PATCH	/api/categories/:id	Admin	Cập nhật danh mục.
DELETE	/api/categories/:id	Admin	Xóa danh mục (từ chối nếu còn sản phẩm active dùng danh mục đó).

5.2. Sản phẩm — /api/products

Method	Đường dẫn	Quyền	Mô tả
GET	/api/products	Công khai	Duyệt sản phẩm với bộ lọc: danh mục, shop, tìm kiếm full-text, sắp xếp (giá/mới/đánh giá), khoảng giá, phân trang.
GET	/api/products/attributes	Công khai	Danh sách thuộc tính (màu, size, trọng lượng...) cho bộ chọn biến thể.
POST	/api/products/attributes/values	Admin	Thêm giá trị thuộc tính.
GET	/api/products/shop/:shopId	Công khai	Sản phẩm active của một cửa hàng.
GET	/api/products/seller/list	Seller/Admin	Danh sách sản phẩm của chính seller (lọc theo trạng thái).
POST	/api/products/quick-create	Seller/Admin	Tạo nhanh sản phẩm để bán trong live (chỉ cần tên, giá, kho, ảnh).
PATCH	/api/products/:id/status	Chủ sản phẩm/Admin	Đổi trạng thái: draft / active / inactive.
DELETE	/api/products/:id	Chủ sản phẩm/Admin	Xóa mềm sản phẩm (status='deleted').
GET	/api/products/:slug	Công khai	Chi tiết một sản phẩm theo slug (cache 30s).

Lưu ý: Thứ tự đăng ký route quan trọng: các đường dẫn cụ thể (attributes, shop, seller) phải đăng ký TRƯỚC ':slug', nếu không Gin sẽ hiểu nhầm 'attributes' là một slug.

5.3. Cửa hàng — /api/shops

Method	Đường dẫn	Quyền	Mô tả
GET	/api/shops	Công khai	Danh sách cửa hàng active, sắp theo doanh số.
GET	/api/shops/:slug	Tùy chọn	Trang công khai của cửa hàng (kèm trạng thái đang theo dõi nếu đăng nhập). :slug có thể là slug, shop ID hoặc seller ID.

Method	Đường dẫn	Quyền	Mô tả
GET	/api/shops/me/info	Seller/Admin	Thông tin cửa hàng của người dùng hiện tại.
GET	/api/shops/me/following	JWT	Danh sách cửa hàng đang theo dõi (phân trang).
GET	/api/shops/:slug/follow-status	JWT	Kiểm tra đã theo dõi cửa hàng hay chưa.
POST	/api/shops/:slug/follow	JWT	Theo dõi cửa hàng (idempotent).
DELETE	/api/shops/:slug/follow	JWT	Bỏ theo dõi.
POST	/api/shops	Seller/Admin	Tạo cửa hàng (mỗi seller một cửa hàng).
PATCH	/api/shops/:slug	Chủ shop/Admin	Cập nhật cửa hàng.

5.4. Ví dụ response (phần data)

```
GET /api/products → 200 OK
{ "data": [ { "id": "p77...", "name": "Cà phê sữa đá",
"slug": "cai-bo-xoi-huu-co", "price": 35000,
"shop_id": "s9...", "in_stock": true } ],
"pagination": { "page": 1, "limit": 20, "total": 134 } }
```

```
GET /api/products/:slug → 200 OK
{ "product": { "id": "p77...", "name": "Cà phê sữa đá",
"price": 35000, "variants": [ {...} ],
"images": [ "https://cdn.../1.webp" ] } }
```

```
POST /api/shops → 403 message: "chỉ tài khoản seller mới tạo
được cửa hàng" (SELLER_ONLY)
```

5.5. Response của TẤT CẢ endpoint danh mục / sản phẩm / cửa hàng

Cột "Thành công" = mã 2xx + data; cột "Các mã lỗi có thể gặp" liệt kê mọi tình huống. 500 và 400 VALIDATION_ERROR ngầm áp dụng mọi nơi; 401 chỉ ghi ở endpoint cần đăng nhập.

Endpoint	Thành công	Các mã lỗi có thể gặp
GET /categories	200 { categories[] }	– (chỉ 500)
POST /categories	201 { category }	401 · 403 ADMIN_ONLY · 422 VALIDATION_ERROR
GET /categories/:slug	200 { category }	404 CATEGORY_NOT_FOUND
PATCH /categories/:id	200 { category }	401 · 403 ADMIN_ONLY · 404 CATEGORY_NOT_FOUND
DELETE /categories/:id	204 (data:null)	401 · 403 ADMIN_ONLY · 409 CATEGORY_IN_USE
GET /products	200 { data[], pagination }	– (chỉ 500)
GET /products/attributes	200 { attribute_types[] }	– (chỉ 500)
POST /products/attributes/values	200 { value }	401 · 403 ADMIN_ONLY · 422 VALIDATION_ERROR

Endpoint	Thành công	Các mã lỗi có thể gặp
GET /products/shop/:shopId	200 { data[], pagination }	404 SHOP_NOT_FOUND
GET /products/seller/list	200 { data[], pagination }	401 · 403 SELLER_ONLY
POST /products/quick-create	201 { product }	401 · 403 SELLER_ONLY · 422 VALIDATION_ERROR
PATCH /products/:id/status	200 { product }	401 · 403 NOT_PRODUCT_OWNER · 404 PRODUCT_NOT_FOUND
DELETE /products/:id	204 (data:null)	401 · 403 NOT_PRODUCT_OWNER · 404 PRODUCT_NOT_FOUND
GET /products/:slug	200 { product: variants, images }	404 PRODUCT_NOT_FOUND
GET /shops	200 { shops[] }	– (chỉ 500)
GET /shops/:slug	200 { shop, is_following }	404 SHOP_NOT_FOUND
GET /shops/me/info	200 { shop }	401 · 403 SELLER_ONLY
GET /shops/me/following	200 { data[], count }	401 UNAUTHORIZED
GET /shops/:slug/follow-status	200 { is_following }	401 · 404 SHOP_NOT_FOUND
POST /shops/:slug/follow	200 { followed:true }	401 · 404 SHOP_NOT_FOUND
DELETE /shops/:slug/follow	200 { followed:false }	401 · 404 SHOP_NOT_FOUND
POST /shops	201 { shop }	401 · 403 SELLER_ONLY · 409 SHOP_EXISTS · 422 VALIDATION_ERROR
PATCH /shops/:slug	200 { shop }	401 · 403 NOT_SHOP_OWNER · 404 SHOP_NOT_FOUND

Lưu ý: ADMIN_ONLY cho danh mục/giá trị thuộc tính (adminMw); SELLER_ONLY cho tạo shop, quick-create, seller/list (sellerMw); NOT*_OWNER cho thao tác lên tài nguyên không phải của mình. Xem mục 1.4 để hiểu ba tầng quyền này.

6. API Giỏ hàng, Đơn hàng & Coupon

6.1. Giỏ hàng — /api/cart

Toàn bộ nhóm này yêu cầu đăng nhập. Giỏ hàng chứa hai loại item: từ catalog (variant thường) và từ phiên live (sản phẩm snapshot trong live).

Method	Đường dẫn	Quyền	Mô tả
GET	/api/cart	JWT	Lấy giỏ hàng kèm tóm tắt (tổng số lượng, tổng tiền, tổng tiết kiệm) — chỉ tính item đang được chọn.
POST	/api/cart/items	JWT	Thêm item theo variant_id (snapshot tên, shop, giá).
POST	/api/cart/items/from-live	JWT	Thêm item từ sản phẩm trong phiên live.
DELETE	/api/cart/items/selected	JWT	Xóa các item đang chọn (đăng ký TRƯỚC /:id để tránh lỗi định tuyến).
PATCH	/api/cart/select-all	JWT	Chọn/bỏ chọn tất cả item.
PATCH	/api/cart/items/:id/qty	JWT	Cập nhật số lượng (1-999), trả về item đã cập nhật.
PATCH	/api/cart/items/:id/select	JWT	Chọn/bỏ chọn một item.
DELETE	/api/cart/items/:id	JWT	Xóa một item.

6.2. Đơn hàng — /api/orders

Giới hạn 20 lần / phút / người để chống lạm dụng token đánh cắp.

Method	Đường dẫn	Quyền	Mô tả
POST	/api/orders	JWT	Đặt hàng một sản phẩm từ phiên live (có khóa kho qua Redis, áp coupon, trừ kho).
POST	/api/orders/checkout	JWT	Thanh toán giỏ hàng (nhiều item, một bản ghi đơn). Hỗ trợ nhiều coupon, snapshot địa chỉ giao hàng.
GET	/api/orders/my/list	JWT	Danh sách đơn của người dùng (phân trang).
GET	/api/orders/:id	Người mua/Admin	Chi tiết đơn; nếu không phải chủ đơn trả về 404 (phòng vệ BOLA).

6.3. Coupon — /api/coupons

Đây là các endpoint hướng người mua (khám phá & xác thực mã). Việc tạo coupon trong live nằm ở nhóm /api/live.

Method	Đường dẫn	Quyền	Mô tả
GET	/api/coupons/available	JWT	Coupon toàn nền tảng (session_id rỗng), hiển thị ở mục 'Tropia Voucher'.
GET	/api/coupons/shop/:shopId	JWT	Coupon thuộc các phiên live của một cửa hàng.

Metho d	Đường dẫn	Quyền	Mô tả
POST	/api/coupons/validate	JWT	Kiểm tra mã trước khi thanh toán (hết hạn, đã dùng, đủ giá trị tối thiểu, giới hạn lượt, mức giảm tối đa).

Lưu ý: Trường unit_price trong đơn hàng có ý nghĩa kép (đặc thù kế thừa từ backend Node.js cũ): với đơn từ live đặt 1 sản phẩm, nó là đơn giá; với đơn từ giỏ hàng, nó là tổng phụ (subtotal). Mẫu email biên nhận dựa vào quy ước này.

6.4. Ví dụ response (phần data)

```
GET /api/cart → 200 OK
{ "items": [ { "id":"ci1...", "product_id":"p77...",
"name":"Cà phê bột 300g", "price":35000,
"quantity":2, "selected":true } ],
"summary": { "selected_count":1, "subtotal":70000 } }
```

```
POST /api/orders/checkout → 201 Created
{ "order": { "id":"o55...", "status":"pending", "total":70000,
"items":[ { "product_id":"p77...", "quantity":2,
"unit_price":70000 } ],
"shipping_address": { "name":"Văn A", "phone":"090...",
"address":"Q1, TP.HCM" } } }
```

```
POST /api/orders → 409 message: "sàn phẩm đã hết hàng" (OUT_OF_STOCK)
GET /api/orders/:id → 404 message: "không tìm thấy đơn hàng" (ORDER_NOT_FOUND)
```

Lưu ý: Đặt hàng khi hết kho trả 409 OUT_OF_STOCK; xem đơn không phải của mình trả 404 ORDER_NOT_FOUND (phòng vệ BOLA), KHÔNG phải 403 — để không lộ rằng đơn đó có tồn tại.

6.5. Response của TẤT CẢ endpoint giỏ hàng / đơn hàng / coupon

Cả nhóm cần đăng nhập nên 401 áp dụng cho mọi dòng. 500 và 400 VALIDATION_ERROR ngầm áp dụng mọi nơi.

Endpoint	Thành công	Các mã lỗi có thể gặp
GET /cart	200 { items[], summary }	401 UNAUTHORIZED
POST /cart/items	201 { item }	401 · 404 PRODUCT_NOT_FOUND · 409 OUT_OF_STOCK
POST /cart/items/from-live	201 { item }	401 · 404 PRODUCT_NOT_FOUND · 409 OUT_OF_STOCK
DELETE /cart/items/selected	204 (data:null)	401 UNAUTHORIZED
PATCH /cart/select-all	200 { items[] }	401 UNAUTHORIZED
PATCH /cart/items/:id/qty	200 { item }	401 · 404 ITEM_NOT_FOUND · 422 (qty 1-999)
PATCH /cart/items/:id/select	200 { item }	401 · 404 ITEM_NOT_FOUND
DELETE /cart/items/:id	204 (data:null)	401 · 404 ITEM_NOT_FOUND

Endpoint	Thành công	Các mã lỗi có thể gặp
POST /orders	201 { order }	401 · 404 PRODUCT_NOT_FOUND · 409 OUT_OF_STOCK · 422 · 429 RATE_LIMITED
POST /orders/checkout	201 { order }	401 · 409 OUT_OF_STOCK · 422 EMPTY_SELECTION · 429 RATE_LIMITED
GET /orders/my/list	200 { orders[], total }	401 UNAUTHORIZED
GET /orders/:id	200 { order }	401 · 404 ORDER_NOT_FOUND (BOLA)
GET /coupons/available	200 { data[] }	401 UNAUTHORIZED
GET /coupons/shop/:shopId	200 { data[] }	401 · 404 SHOP_NOT_FOUND
POST /coupons/validate	200 { valid, discount_amount, final_total }	401 · 422 MIN_ORDER_NOT_MET / COUPON_EXPIRED / COUPON_USED

7. API Thanh toán

Nhóm `/api/payment` tích hợp ba cổng thanh toán Việt Nam: MoMo, VNPay và ZaloPay. Mỗi cổng có endpoint khởi tạo (cần đăng nhập) và endpoint callback/IPN (cổng gọi về, công khai nhưng xác minh chữ ký). Khởi tạo thanh toán giới hạn 10 lần / phút / người.

Meth od	Đường dẫn	Quyền	Mô tả
POST	<code>/api/payment/momo</code>	JWT	Khởi tạo thanh toán MoMo, trả về <code>payUrl/deeplink/QR</code> .
GET	<code>/api/payment/momo/callback</code>	Công khai	MoMo chuyển hướng trình duyệt về sau khi thanh toán; xác nhận đơn và redirect.
POST	<code>/api/payment/momo/ipn</code>	Công khai	Webhook MoMo xác minh chữ ký (13 trường HMAC-SHA256) rồi xác nhận đã thanh toán.
POST	<code>/api/payment/vnpay</code>	JWT	Khởi tạo URL thanh toán VNPay (ký HMAC-SHA512, bắt IP người dùng).
GET	<code>/api/payment/vnpay/return</code>	Công khai	VNPay chuyển hướng về; xác minh chữ ký rồi xác nhận đơn.
POST	<code>/api/payment/zalopay</code>	JWT	Khởi tạo thanh toán ZaloPay (ký HMAC-SHA256 với key1).
POST	<code>/api/payment/zalopay/callback</code>	Công khai	Webhook ZaloPay xác minh MAC bằng key2; phải trả về <code>return_code=1</code> khi thành công.

7.1. Luồng xác nhận thanh toán

- 1. Client khởi tạo thanh toán → backend kiểm tra quyền sở hữu đơn (BOLA) và đơn chưa thanh toán → gọi cổng để lấy URL.
- 2. Người dùng thanh toán trên cổng → cổng gọi về qua hai đường: chuyển hướng trình duyệt (callback/return) và webhook server-to-server (IPN/callback).
- 3. Backend xác minh chữ ký, gọi `ConfirmPayment`: đánh dấu đơn 'paid', dọn các item đã đặt khỏi giỏ, và phát sự kiện 'payment.success'.
- 4. Worker nhận sự kiện và gửi email biên nhận chi tiết (kèm bảng sản phẩm và địa chỉ giao hàng).

Lưu ý: `ConfirmPayment` idempotent: nếu nhận callback/IPN trùng lặp, lần thứ hai là no-op và không gửi lại email biên nhận.

7.2. Ví dụ response (phần data)

```
POST /api/payment/momo → 200 OK
{ "pay_url": "https://test-payment.momo.vn/...",
  "deeplink": "momo://...", "qr_code_url": "https://.../qr/...",
  "order_id": "o55...", "amount": 56000 }
```

```
POST /api/payment/vnpay → 200 OK
{ "pay_url": "https://sandbox.vnpayment.vn/...?vnp_Amount=5600000&...",
  "order_id": "o55..." }
```

```
POST /api/payment/zalopay/callback → 200 OK
{ "return_code": 1, "return_message": "success" }
// chữ ký sai → vñn 200 nhưng { "return_code": -1, "return_message": "mac not equal" }
```

Lưu ý: Các endpoint callback/return/ipn do CỔNG gọi (không phải client) và phần lớn nằm trong danh sách bỏ qua hoặc trả non-JSON, nên KHÔNG bị bọc envelope: chúng hầu như luôn trả 200/302 để cổng không gọi lại vô hạn; trạng thái thành/bại nằm trong body (return_code) hoặc tham số redirect. Số tiền VNPay nhân 100 nên 56.000đ thành 5600000 trong URL.

7.3. Response của TẤT CẢ endpoint thanh toán

Endpoint khởi tạo (POST momo/vnpay/zalopay) cần đăng nhập (401) và giới hạn 10 lần/phút (429). Endpoint callback/return/ipn do CỔNG gọi, hầu như luôn 200/302 (không bọc envelope) — trạng thái thật nằm trong body.

Endpoint	Thành công	Các mã lỗi có thể gặp
POST /payment/momo	200 { pay_url, deeplink, qr_code_url }	401 · 403 NOT_ORDER_OWNER · 404 ORDER_NOT_FOUND · 409 ALREADY_PAID · 429 RATE_LIMITED
GET /payment/momo/callback	302 redirect (không bọc)	302 redirect kèm trạng thái thất bại
POST /payment/momo/ipn	200 { message:ok } (không bọc)	200 + INVALID_SIGNATURE (cùng yêu cầu 200)
POST /payment/vnpay	200 { pay_url (vnp_Amount×100) }	401 · 403 NOT_ORDER_OWNER · 404 · 409 ALREADY_PAID · 429 RATE_LIMITED
GET /payment/vnpay/return	302 redirect (không bọc)	302 redirect kèm trạng thái thất bại
POST /payment/zalopay	200 { order_url, zp_trans_token }	401 · 403 NOT_ORDER_OWNER · 404 · 409 ALREADY_PAID · 429 RATE_LIMITED
POST /payment/zalopay/callback	200 { return_code:1 } (không bọc)	200 { return_code:-1, "mac not equal" }

8. API Tải ảnh & Webhook SRS

8.1. Tải ảnh — /api/upload

Chỉ bật khi đã cấu hình Cloudflare R2. Mỗi ảnh tối đa 5MB, định dạng JPG/PNG/WEBP. Các endpoint sản phẩm/biến thể/shop/live đều kiểm tra quyền sở hữu (BOLA) trước khi cho ghi đè ảnh.

Meth od	Đường dẫn	Quyền	Mô tả
POST	/api/upload/avatar	JWT	Tải ảnh đại diện người dùng.
POST	/api/upload/product	Seller/Admin	Tải ảnh sản phẩm (tối đa 10 file); kiểm tra seller sở hữu product_id.
POST	/api/upload/variant	Seller/Admin	Tải ảnh biến thể (tối đa 5 file).
POST	/api/upload/shop	Seller/Admin	Tải ảnh banner cửa hàng.
POST	/api/upload/live	Seller/Admin	Tải ảnh bìa phiên live.
POST	/api/upload/temp	Seller/Admin	Tải ảnh tạm.

```
POST /api/upload/product → 200 OK (phản data)
{ "urls": [ "https://cdn.tropia.vn/products/p77/1.webp",
"https://cdn.tropia.vn/products/p77/2.webp" ] }
POST /api/upload/avatar → 200 OK
{ "url": "https://cdn.tropia.vn/avatars/8c1d.webp" }
```

8.2. Webhook SRS — /api/srs

Các endpoint này do SRS gọi, không phải client. Tất cả được bảo vệ bằng tham số bí mật chung ?secret=... (so sánh constant-time). Production bắt buộc đặt secret, nếu không backend từ chối khởi động.

Meth od	Đường dẫn	Quyền	Mô tả
POST	/api/srs/on_publish	SRS	Stream bắt đầu → đánh dấu phiên 'live' (từ chối stream key lạ).
POST	/api/srs/on_unpublish	SRS	Stream ngắt → đóng dấu thời điểm kết thúc (chưa đổi trạng thái để chịu được RTMP dứt rồi nối lại).
POST	/api/srs/on_play	SRS	Người xem kết nối (hiện chỉ trả OK).
POST	/api/srs/on_stop	SRS	Người xem ngắt (hiện chỉ trả OK).
POST	/api/srs/on_dvr	SRS	Ghi hình DVR xong → phát sự kiện 'recording.created' cho worker xử lý VOD.

Lưu ý: Webhook SRS trả về một CON SỐ trần (KHÔNG bọc JSON, KHÔNG bọc envelope — nằm trong skipEnvelope): 0 = cho phép phát, khác 0 (vd 1) = từ chối (stream key lạ hoặc sai secret → SRS ngắt luồng). Đây là quy ước HTTP callback của SRS, không theo khuôn envelope như các API thường.

8.3. Endpoint hệ thống

Meth od	Đường dẫn	Quyền	Mô tả
GET	/health	Công khai	Kiểm tra sống (liveness probe). KHÔNG bọc envelope.
GET	/metrics	Token	Số liệu Prometheus (text), bảo vệ bằng METRICS_TOKEN. KHÔNG bọc envelope.
GET	/docs/openapi.yaml	Token	Đặc tả OpenAPI 3.1 (file YAML), gate bằng DOCS_TOKEN ở production. KHÔNG bọc envelope.

8.4. Response của TẤT CẢ endpoint tải ảnh, webhook & hệ thống

Upload cần đăng nhập (401), giới hạn dung lượng/định dạng (400/413). Webhook SRS trả SỐ TRẦN (không bọc envelope). Endpoint hệ thống cũng không bọc envelope.

Endpoint	Thành công	Các mã lỗi có thể gặp
POST /upload/avatar	200 { url }	401 · 400 INVALID_FILE · 413 PAYLOAD_TOO_LARGE
POST /upload/product	200 { urls[] }	401 · 403 SELLER_ONLY · 403 NOT_PRODUCT_OWNER · 400 INVALID_FILE
POST /upload/variant	200 { urls[] }	401 · 403 SELLER_ONLY · 403 NOT_PRODUCT_OWNER · 400 INVALID_FILE
POST /upload/shop	200 { url }	401 · 403 SELLER_ONLY · 403 NOT_SHOP_OWNER · 400 INVALID_FILE
POST /upload/live	200 { url }	401 · 403 SELLER_ONLY · 403 NOT_STREAM_OWNER · 400 INVALID_FILE
POST /upload/temp	200 { url }	401 · 403 SELLER_ONLY · 400 INVALID_FILE
POST /srs/on_publish	200 → "0" (cho phép)	200 → "1" (key lạ / sai secret → SRS ngắnt)
POST /srs/on_unpublish	200 → "0"	200 → khác 0 nếu sai secret
POST /srs/on_play · on_stop	200 → "0"	200 → khác 0 nếu sai secret
POST /srs/on_dvr	200 → "0" (phát recording.created)	200 → khác 0 nếu sai secret
GET /health	200 { status:"ok" }	503 { status:"degraded" } (DB/Redis chắtt)
GET /metrics	200 text Prometheus	401 UNAUTHORIZED (sai METRICS_TOKEN)
GET /docs/openapi.yaml	200 file YAML	401 UNAUTHORIZED (sai DOCS_TOKEN)

9. Worker nền & hệ thống sự kiện

Backend dùng Redis Streams làm hàng đợi sự kiện nhẹ (thay cho RabbitMQ). API phát sự kiện, cmd/worker tiêu thụ và xử lý. Mỗi loại sự kiện là một stream riêng; consumer group cho giao hàng ít nhất một lần và tự thử lại khi lỗi.

Sự kiện	Người phát	Xử lý của worker
user.registered	verify-otp	Gửi email chào mừng
auth.email_otp	register / resend	Gửi email mã OTP
auth.password_reset	forgot-password	Gửi email link đặt lại mật khẩu
order.created	đặt hàng / checkout	Gửi email xác nhận (với COD là biên nhận luôn)
payment.success	ConfirmPayment	Gửi email biên nhận chi tiết
order.cancelled	hủy đơn	Ghi log (mở rộng sau)
recording.created	SRS on_dvr	Ghép phụ đề chat + overlay → MP4 → tải lên R2

9.1. Xử lý VOD (xem lại phiên live)

Đây là phần phức tạp nhất của worker. Khi nhận 'recording.created', worker:

- **1.** Tải bản ghi FLV và nạp timeline (chat + sự kiện host) từ DB.
- **2.** Dựng phụ đề chat dạng ASS (libass) và render các overlay PNG (ghim sản phẩm, coupon, chip bot, danh sách sản phẩm).
- **3.** Dùng FFmpeg ghép tất cả vào video, mã hóa lại ra MP4 (đây là đường chậm, ~1x thời gian thực).
- **4.** Tải MP4 lên Cloudflare R2 và lưu URL vào live_sessions.vod_mp4_url để app phát lại.

Ngoài ra worker chạy một bộ quét DVR định kỳ để xóa file FLV mở còi (do remux lỗi hoặc worker crash), tránh đầy ổ đĩa.

10. Tóm tắt cho người mới

Nếu bạn mới vào dự án, hãy ghi nhớ các ý chính sau:

- **Video tách khỏi nghiệp vụ:** SRS lo hoàn toàn việc nhận và phát video qua các giao thức chuẩn (RTMP/SRT/WHIP đẩy lên, HLS/FLV/WHEP phát xuống). Backend Go chỉ quản lý metadata phiên, sinh URL và nhận webhook từ SRS — nó không 'chạm' vào từng khung hình video.
- **Realtime đi qua hai đường:** video qua HLS (~5-8s trễ), còn chat/số liệu/coupon đi qua WebSocket (gần tức thì). App lắng nghe một kênh WebSocket duy nhất cho mỗi phiên.
- **Mọi response JSON đều bọc envelope:** hình dạng thật luôn là { Result, StatusCode, StatusMess, status, message, data }. Payload của handler nằm trong data; lỗi → data:null + thông điệp ở message (mục 1.3).
- **Phân quyền ba tầng:** buyer → seller → admin trong JWT claim; guard authMw/sellerMw/adminMw kiểm tra VAI TRÒ, còn service kiểm tra QUYỀN SỞ HỮU (mục 1.4).
- **Nghiệp vụ theo module rõ ràng:** auth, live, catalog (sản phẩm/danh mục), shops, commerce (giỏ/đơn/coupon), payment, upload, srs. Mỗi module có repository (truy vấn DB), service (nghiệp vụ) và handler (HTTP).

Để bắt đầu chạy thử cục bộ: khởi động hạ tầng bằng docker compose up -d trong thư mục infra, sao chép .env.example thành .env.development, chạy go run ./cmd/api và go run ./cmd/worker, rồi go run ./cmd/seed để nạp dữ liệu mẫu (mật khẩu mọi tài khoản test là Password123).